Lab Report

Jobsheet 6

Inheritance

Herconary Angga
244107020215

Experiment 1:

Output:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
        x cannot be resolved or is not a field
        y cannot be resolved or is not a field
        The method getNilai() is undefined for the type ClassB

        at Extends.Percobaan1.main(Percobaan1.java:7)
```

Question:

1. In experiment 1 above, the program that was run ran an error, then fix it so that the program can be run and not error!

```java
package Extends;

public class ClassB extends ClassA {
    public int z;

    public void getNilaiZ() {
        System.out.println("nilai z: " + z);
    }

    public void getJumlah() {
        System.out.println("jumlah: " + (x + y + z));
    }
}
```

```
nilai x: 20
nilai y: 30
nilai z: 40
jumlah: 90
```

Added extends when declaring ClassB

2. Explain what caused the program in experiment 1 when there was an error!

   There are no variable x and y yet since the ClassB wasn't a subclass of ClassA before.

Experiment 2:

Output:

```
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
        The method setX(int) is undefined for the type ClassB
        The method setY(int) is undefined for the type ClassB
        The method getNilai() is undefined for the type ClassB

        at RightOfAccess.Percobaan2.main(Percobaan2.java:6)
```

Questions:

1. In experiment 2 above, the program ran an error, then fix it so that the program can be run and not error!

```java
package RightOfAccess;

public class ClassA {
    protected int x, y;

    public void setX(int x) {
        this.x = x;
    }

    public void setY(int y) {
        this.y = y;
    }

    public void getNilai() {
        System.out.println("Nilai x: " + x);
        System.out.println("Nilai y: " + y);
    }
}
```

```java
package RightOfAccess;

public class ClassB extends ClassA {
    private int z;

    public void setZ(int z) {
        this.z = z;
    }

    public void getNilaiZ() {
        System.out.println("nilai z: " + z);
    }

    public void getJumlah() {
        System.out.println("jumlah: " + (x + y + z));
    }
}
```

```
Nilai x: 20
Nilai y: 30
nilai z: 5
jumlah: 55
```

Changed x and y's access modifier to protected, and making ClassB a subclass of ClassA.

2. Explain what caused the program in experiment 1 when there was an error!

   x and y's access modifiers are private, so none other than their own class can access the variable. And ClassB wasn't a subclass of ClassA which makes x and y are unidentified by in ClassB.

Experiment 3:

Output:

```
Volume Tabung adalah: 942.0
```

Questions:

1. Describe the "super" function in the following program pieces in the Tubes class!
   super.phi = phi;
   super.r = r;

   It's to initialize the value of phi and r through Tabung, since they are originally Bangun's variables, while Tabung is a subclass of Bangun.

2. Explain the functions of "super" and "this" in the following program snippets in the Tube!
   System.out.println("Volume Tabung adalah: " + (super.phi * super.r * super.r * this.t));

   Super refers to its parent class's variables, which are phi and r. And this refers to that class's own variables, which is t.

3. Explain why the Tube class doesn't declare the "phi" and "r" attributes, but the class can access them!

   Because Tube's the subclass of Bangun Class, and the variables in the Bangun Class were not using private access modifier, so automatically Tube can already access Bangun's variables.

Experiment 4:

Output:

```
Konstruktor A dijalankan
Konstruktor B dijalankan
Konstruktor C dijalankan
```

Questions:

1. In experiment 4, specify which class belongs to superclasses and subclasses, then explain why!

   Class A → Super Class of ClassB
   Class B → Super Class of ClassC

2. Change the contents of the default ClassC constructor as follows:

```
public class ClassC extends ClassB{
    ClassC(){
    super();
    System.out.println("konstruktor C dijalankan");
    }
}
```

   Add the word super() in the Garden row in its default constructor. Try running the Experiment4 class again and there is no difference in the output!

```
Konstruktor A dijalankan
Konstruktor B dijalankan
Konstruktor C dijalankan
```

3. The contents of the default ClassC constructor can be as follows:

```
12  public class ClassC extends ClassB{
13      ClassC(){
14          System.out.println("konstruktor C dijalankan");
            super();
16      }
17  }
```

When changing the position of super() in the second row in its default constructor there is an error. Then return the first line super() as before, then the error will disappear. Notice the output results when the Experiment4 class is run. Why can the output be displayed as follows when the test object is institutionalized from the ClassC class.

```
Output - Percobaan4 (run)
run:
konstruktor A dijalankan
konstruktor B dijalankan
konstruktor C dijalankan
BUILD SUCCESSFUL (total time: 0 seconds)
```

Explain how the constructor runs when the test object is created!

Java automatically runs the constructor whatsoever whether we directly write super(); or not. So writing the super(); basically still calls ClassB's constructor.

4. What is the super() function in the below program snippet in ClassC!

```
12  public class ClassC extends ClassB{
13      ClassC(){
14          System.out.println("konstruktor C dijalankan");
            super();
16      }
17  }
```

To call the ClassB's constructor from the ClassC.

Assignment:

Code:

```java
package Assignment;

public class Pegawai {
    protected String nip, nama, alamat;

    public Pegawai(String nip, String nama, String alamat) {
        this.nip = nip;
        this.nama = nama;
        this.alamat = alamat;
    }

    public String getNama() {
        return nama;
    }

    public int getGaji() {
        return 0;
    }
}
```

```java
package Assignment;

public class Dosen extends Pegawai {
    protected int jumlahSKS, tarifSKS;

    public Dosen(String nip, String nama, String alamat, int jumlahSKS, int tarifSKS) {
        super(nip, nama, alamat);
        this.jumlahSKS = jumlahSKS;
        this.tarifSKS = tarifSKS;
    }

    public void setJumlahSKS(int jumlahSKS) {
        this.jumlahSKS = jumlahSKS;
    }

    public int getGaji() {
        return jumlahSKS * tarifSKS;
    }
}
```

```java
package Assignment;

public class DaftarGaji {
    protected Pegawai[] listPegawai;
    protected int pegawaiCounter = 0;

    public DaftarGaji(int jumlah) {
        listPegawai = new Pegawai[jumlah];
    }

    public void addPegawai(Pegawai pegawai) {
        listPegawai[pegawaiCounter] = pegawai;
        pegawaiCounter++;
    }

    public void printSemuaGaji() {
        for (int i = 0; i < pegawaiCounter; i++) {
            System.out.println(listPegawai[i].getNama() + ": Rp. " + listPegawai[i].getGaji());
        }
    }
}
```

```java
package Assignment;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        DaftarGaji daftarGaji = new DaftarGaji(jumlah:2);


        Dosen d1 = new Dosen(nip:"123", nama:"Snock", alamat:"Sorna", jumlahSKS:10, tarifSKS:120000);
        Dosen d2 = new Dosen(nip:"234", nama:"Zeb", alamat:"BioSyn", jumlahSKS:8, tarifSKS:320000);

        daftarGaji.addPegawai(d1);
        daftarGaji.addPegawai(d2);
        daftarGaji.printSemuaGaji();
    }
}
```

Output:

```
Snock: Rp. 1200000
Zeb: Rp. 2560000
```